AgriSense BW: 3-Day Team Task Plan

Team: Eddie, Thapelo, Abel, Phillip and Eazy

AgriSense BW is an AI-powered smart agriculture platform for Botswana with two modules, crops and livestock, joined by one early-warning engine. We have about 3 days, so the plan uses sample or public data and a simple prototype: one simple model per module, one shared alert format, and one clear demo.

Shared rules (everyone reads this first)

One shared folder (Google Drive or GitHub) with these parts: `data`, `models`, `dashboard`, `report`.

Pilot area: choose one district for the whole project, so everyone uses the same place.

Two data files (Eddie makes them, everyone uses them):

- `crop_data.csv`: date, district, rainfall_mm, temperature_c, ndvi (plant greenness), yield_t_ha
- `livestock_data.csv`: date, animal_id, herd_id, district, lat, lon, activity_level, body_temp_c

One alert format (everyone's results end up here):

- district, type (crop or livestock), risk_level (Low, Medium, High), reason, advice_en, advice_tn

Eddie: Data lead

Goal: give the others their data by the end of Day 1.

Steps:

1. On the morning of Day 1, hold a 30-minute team call and explain the three shared rules above.
2. Choose the pilot district with the team.
3. Get rainfall and temperature data from public sources (for example NASA POWER or CHIRPS, or the Botswana Department of Meteorological Services). Put it into `crop_data.csv`.
4. For ndvi and yield, use public values if you find them. If not, make realistic sample values, and say clearly in the report that they are sample data.

5. Real livestock GPS data is very hard to get, so write a small Python script that simulates about 30 animals moving for 30 days. Make most animals normal, and make 3 or 4 behave strangely (for example, very low activity with a high body temperature, or moving far at night). Save it as `livestock_data.csv`.

Hand in: both CSV files, plus one short note explaining each column.

Day 2 and 3: help anyone who is stuck, and write the report sections Problem, Solution overview and Data.

Thapelo: Crop model

Goal: predict crop yield and flag crops at risk.

Receives: `crop_data.csv`

Steps:

1. Load the file with pandas.
2. Train a simple model (linear regression or random forest from scikit-learn) to predict `yield_t_ha` from rainfall, temperature and ndvi.
3. Check it with the test score (R^2 or MAE) and make one chart of predicted against real yield.
4. Rule for risk: if predicted yield is below 70% of the average, it is High; 70% to 90% is Medium; above that is Low.
5. Save the results as `crop_alerts.csv` in the shared alert format (leave the advice columns empty).

Hand in: the notebook, the chart, and `crop_alerts.csv` by the end of Day 2 morning.

Abel: Livestock monitoring

Goal: find animals that behave strangely.

Receives: `livestock_data.csv`

Steps:

1. For each animal and each day, calculate the distance moved from the lat and lon values.
2. Compare each animal with its herd average. Use a simple z-score: if an animal is more than 2 standard deviations away, flag it.
3. Give a reason to each flag: low activity with high body temperature = possible illness; very large movement or movement at night = possible theft or stress.
4. Make one chart or map showing the flagged animals.

5. Save the results as `livestock_alerts.csv` in the shared alert format.

Hand in: the notebook, the chart, and `livestock_alerts.csv` by the end of Day 2 morning.

Phillip: Early-warning engine

Goal: combine everything into one alert list that farmers can understand.

Receives: `crop_alerts.csv` and `livestock_alerts.csv`

Day 1: agree the alert format with Thapelo and Abel (the list above is the starting point), so their files match.

Day 2 steps:

1. Merge both files into one table.
2. Add a disease outbreak rule: if 3 or more animals in the same district are flagged as possibly ill within a few days, create a High outbreak warning for veterinary services.
3. Write advice templates, in simple English and in Setswana, for each risk type and level. Keep each one to one or two sentences.
4. Save the result as `final_alerts.csv`.

Hand in: `final_alerts.csv` and the script, by the end of Day 2.

Eazy: Dashboard and report

Goal: show the results clearly and keep the documents together.

Receives: all the files above (at first, use fake sample values so you can start early).

Day 1: make the layout. The easiest tool is Streamlit (Python), with three tabs: Crops, Livestock, Alerts. Also create the report document with headings: Problem, Solution, Data, Crop Module, Livestock Module, Alert Engine, Results, Limitations, Future Work.

Day 2: connect the real files to the dashboard. Show a map of the animals (flagged ones in red) and a table of the alerts with colours for Low, Medium and High.

Day 3: collect everyone's short summaries into the report, and make 6 to 8 slides.

Everyone on Day 3

Write a half-page summary of your own part: what you built, how it works, what the result was, and one limitation. Send it to Eazy by midday. Then the whole team practises the demo together, and only small fixes are allowed after that.

If time runs short

Drop satellite imagery and leaf-photo disease detection first. Mention them as the roadmap, and keep the core demo working.